Analysis of Household Surveys

Session notes

Ethan Ligon

September 8, 2026

Contents

1 Design, Sampling, and Collection

What a careful analyst should know before touching the data.

1.1 Reading

- Deaton ch. 1 — the whole chapter
- Deaton §1.4 on survey design and §2.2 on what to do about it
- Skim the GLSS7 *Report* methodology section

Deaton is open access (CC BY 3.0 IGO). On the hub it is already in your `reading/` folder; otherwise PDF.

The GLSS7 report is published by the Ghana Statistical Service and is not ours to redistribute: GSS catalogue 97.

1.2 Setup

```
%xmode Plain

import lsms_library as ll
import pandas as pd

WAVE = '2016-17'
```

1.3 Brief description of LSMS Surveys

A timeline of the surveys `lsms_library` knows about: one row per country, one bar per survey wave, so we can see overall temporal coverage — 37 countries, 113 waves, 1985 to the present.

The same page carries a graphical version of the coverage matrix: which features the library grades, wave by wave, and how complete each one is.

`https://claude.ai/code/artifact/7746f698-7b43-49d0-b2fc-22f7a1fb6496`

(Also linked from the workshop page as *LSMS Library Coverage*.)

1.4 Topical coverage of LSMS Surveys

The picture above is built from a table you can query yourself.

```
cov = ll.coverage(refresh="coverage", countries=["GhanaLSS"])
print("features: %d | waves: %d"
      % (cov.feature.nunique(), cov[cov.wave != ""].wave.nunique()))
cov.tier.value_counts().head(4)
```

`declared` means the library wires that feature for that wave; `absent` and `undeclared` mean it does not. This is a *coverage-only* run — it reads the configuration without building anything, which is why it takes a second rather than an hour. The page linked above goes further and reports whether each one actually builds.

So, what is available for a country you care about:

```
cov.head(10)
```

1.5 LSMS_Library is a work in progress

Open source; see `https://github.com/ligon/lsms_library`. The idea is to provide a software *abstraction layer* to interface with LSMS-style surveys, providing a set of common labels, tables, and variables.

Note that this is **not** "harmonization"; we preserve all the detail of the individual surveys. Aggregation across surveys where necessary is left to the analyst.

If you find a problem, something's missing, or have a feature request: `https://github.com/ligon/lsms_library/issues`. I'd really appreciate your help!

1.6 Explore Example Country

I'll use Uganda as my example; it's one of the best developed countries in `lsms_library`. But I encourage you to play with others; just change the string 'Uganda' below.

```
eg = ll.Country('Uganda')
eg.waves
```

What different kinds of features/data are available for this country?

```
eg.data_scheme
```

What does the sample look like?

```
eg.sample()
```

Some actual data:

```
eg.household_roster()
```

1.7 Survey Design

Any household survey has:

Population What is the *population* that is supposed to be represented?

Sample A collection of selected households

Sample Frame What is the practical information we have on the population that we can use to construct the sample?

Sample Weights Proportional to the *ex ante* probability that a given sample household was selected for inclusion in the sample.

Sample Data Actual information collected from actual households in the sample

1.8 Survey Design Example

Consider the Ghana Living Standards Survey (GhanaLSS), and specifically the most recent publicly released wave 7 (2016–17).

Sample Frame A list of private dwellings from the 2010 census. What does this exclude?

Sample Weights Proportional to the inverse of the *ex ante* probability that a given sample household was selected for inclusion in the sample. The weights in `lsms_library` are normalized to have a mean of one, but sometimes instead they're normalized to sum to the estimated size of the population.

1.9 Survey Design Example

Consider the Ghana Living Standards Survey (GhanaLSS), and specifically the most recent publicly released wave 7 (2016–17).

Population

```
glss = ll.Country('GhanaLSS')
glss.population[WAVE].population_statement
```

Sample

```
glss7 = ll.Country('GhanaLSS')[WAVE]
glss7.sample()
```

Sample Frame A list of private dwellings from the 2010 census. What does this exclude?

Sample Weights Proportional to the inverse of the *ex ante* probability that a given sample household was selected for inclusion in the sample. The weights in `lsms_library` are normalized to have a mean of one, but sometimes instead they're normalized to sum to the estimated size of the population.

Sample Data Actual information collected from actual households in the sample. For example, here's some information on the individuals within each household in the GhanaLSS 2016-17 data:

```
glss7.household_roster()
```

There are many different similar tables available; to see them

```
glss.features
```

1.10 Choosing a Population

In designing a population survey, one has to first choose a population. We'll be focused on *household* surveys within a particular country.

That sounds like a pretty good start: for example, all households in Ghana. But almost immediately edge cases turn up.

- First, what's a household? Usual definition: a set of people who "share a cooking pot"
- What about people in Ghana who don't live in households?
  - Convicts who live in prisons
  - Students who live in dormitories
  - Nuns who live in Nunneries

1.11 Choosing a Population: typical solution

"Private households" (so we're excluding the above)

1.12 Choosing a Survey Frame

If we've settled on "private households", we have to figure out a way to actually somehow enumerate all the private households in a country so we can make a random selection.

- Possibilities
  - Use a census (the usual LSMS approach)
  - Use remote sensing and machine learning to try to identify private dwellings (Togo a recent example)
- Problems
  - Census is usually out of date
  - Remote sensing is highly error prone
- Leads to demand for ground-truthing

1.13 Census approach to enumerating households

Typical approach:

1. Divide entire country into "Enumeration Areas" (EAs).
2. Send a team to each EA; have them identify/map all residential structures (separate dwellings, apartment blocks, compounds).
3. Find someone who lives in each structure; if it's a private dwelling, list all who live there.

1.14 Census approach to enumerating households

Step 3 in full:

Find someone who lives in each structure; if it's a private dwelling, list all who live there. If it's an apartment block, get access, and list all who live in each apartment. If it's a compound with multiple dwellings: list regular occupants of each. Are cooking duties shared across dwellings? If so it's a single household; otherwise multiple HHs.

1.15 Two-Stage Cluster Based Sampling

At the time a census is conducted, it's usually the most reliable "ground-truth"; it provides a list of private households, you could just randomly select. But:

- Census goes out of date almost immediately; new construction, people move, etc.

First Stage: Sample EAs Instead, take a random sample of EAs. Then, *just for these sampled areas*, send a team to update the census listing, and get an up-to-date list of all households in just the *selected* EAs.

ond Stage: Sample Households For each sampled EA, randomly sample the households within it.

1.16 Example: Two-Stage Sampling Weights

Guinea-Bissau provides a fairly simple example. It stratifies by region (see below); within a region it uses a simple two-stage sample. Consider the region Bafata; here is a map of the sample EAs chosen in the first stage.

```
ll.coordinate_map('Guinea-Bissau', where={'Region': 'bafata'})
```

1.17 Sampling Weights

We can always compute statistics (mean, variance, etc.) that are valid for the *sample*; but usually we'd like to use the sample to say something about the *population*. For this we need to know the probability π_i that a particular sample household i was included in the sample *ex ante*.

Once we know these we can compute statistics from a sample of size N.

For the sample:

Mean $\bar{x} = \frac{1}{N} \sum_{i \in \text{Sample}} x_i$

Variance $\text{Var}(x) = \frac{1}{N} \sum_{i \in \text{Sample}} x_i^2 - \bar{x}^2$

1.18 Sampling Weights

For the population

Let $\hat{\bar{X}} = \frac{\sum_{i \in \text{Sample}} x_i/\pi_i}{\sum_{i \in \text{Sample}} 1/\pi_i}$ (this is the Hájek estimator).

Mean $\bar{X} = \mathbb{E}\left[\hat{\bar{X}}\right] + O(1/N)$

Variance $\mathbb{E}(X - \bar{X})^2 = S^2 = \mathbb{E}\left[\frac{\sum_{i\in\text{Sample}}(x_i - \hat{\bar{X}})^2/\pi_i}{\sum_{i\in\text{Sample}} 1/\pi_i}\right] + O(1/N)$

Takeaway: To draw inferences about the population we need to know sampling probabilities π_i.

1.19 Sub-Populations

Conducting a large-scale household survey is expensive! Sometimes you'd like to draw inferences not just about the original population (e.g., all the private households in Ghana), but also for subpopulations: e.g.,

- Particular regions
- Rural vs. Urban

The GLSS: Later rounds of the GhanaLSS are designed to do *both*; for example, to make population statements about the urban population of the Ashanti region.

1.20 Stratification

The most statistically efficient way to do inference for sub-populations is via **stratification**: Partition the overall population into different parts, and (effectively) treat each one as it's own sampling problem.

- Let N_s be the size of the sample for stratum s, and n_s be the size of the population in the stratum.

1.21 Strata Weights

If $N_s/n_s \neq N/n$ (the sample drawn in stratum s isn't proportional to the share of s in the population) then we've violated the maxim that every household in the overall population has an equal chance of being selected. To correct this, construct strata weights

$$v_s = \frac{N_s/n_s}{N/n}$$

to correct for the fact that a household in stratum s has a probability of being selected into the sample which must be adjusted by the factor v_s.

1.22 Two-Stage Sampling Weights

In a Two-Stage Clustered sampling scheme, the probability of a particular household i who lives in EA c_i being included in the sample depends on two other probabilities:

- The probability of EA (cluster) c being selected in the first stage, say p_{c_i};
- *Conditional* on c_i being selected, the probability of i being selected from all the other households in the cluster, say r_i.

So: $\pi_i = p_{c_i} r_i$ in a two stage clustering scheme.

1.23 Example: Two-Stage Sampling Weights

Back to our Bafata example.

```
ll.coordinate_map('Guinea-Bissau', size='weight', where={'Region': 'bafata'
```

```
s = ll.Country("Guinea-Bissau").sample().reset_index()
b = s[s["strata"] == "Bafata"]

cl = b.groupby("v").agg(w=("weight", "first"),
                        urban=("Rural", "first"),
                        take=("weight", "size"))
cl["w"] = cl["w"].astype(float)

print("clusters  :", len(cl))
print("households:", len(b))
print("take      :", dict(cl["take"].value_counts()))
```

Fifty clusters, 598 households, and forty-eight of the fifty contain exactly twelve households — the two that hold eleven are non-response, not design. This is the textbook object: draw clusters, take a fixed number from each.

```
cv = b.groupby("v")["weight"].std().fillna(0) / b.groupby("v")["weight"].me
print("clusters whose weight varies inside them:", int((cv > 1e-9).sum()),
print("weights: min %.3f  median %.3f  max %.3f" % (cl.w.min(), cl.w.median
```

The weight is constant within every cluster, but not *across* clusters.

- Everyone in cluster selected with equal probability
- But weights differ *across* clusters, which tells us different cluster are selected with *different* probabilities.

1.24 Inferring sample design from weights

For household i in cluster c_i, a two-stage design factors the probability of selection:

$$\pi_i = \underbrace{r_{c_i}}_{\text{cluster chosen}} \times \underbrace{\frac{b_{c_i}}{M_{c_i}}}_{\text{chosen within it}}$$

b_{c_i} is the number of households in the cluster selected (twelve); M_{c_i} is the size of the cluster. The weight is the reciprocal:

$$w_i \propto \frac{1}{\pi_i} = \frac{M_i}{p_i\, b_i}$$

One equation, two unknowns. We see w and b; we want p and M.

1.25 Inferring sample design from weights

```
print("sum of weights over the whole survey:", round(s["weight"].sum(), 1))
print("households in the survey             :", len(s))
```

Equal — the library normalizes weights to mean 1, so they carry *relative* probabilities and nothing about population totals. Everything below is a ratio.

Perhaps the fifty clusters were drawn by **simple random sampling** from Bafata's enumeration areas, each equally likely. Then $p_c = N/n$ is the same for all of them and the identity collapses:

$$w_i \propto \frac{M_i}{b_i}$$

A household in a large EA is less likely to be drawn, so it stands for more people.

```
cl["M_rel"] = cl["w"] * cl["take"]
cl["M_rel"] = cl["M_rel"] / cl["M_rel"].median()
```

```
print(cl["M_rel"].describe(percentiles=[.05, .25, .5, .75, .95]).round(2).t
```

Now the sanity check, which is the point of the whole section. Half the clusters fall between 0.89 and 1.04 times the median. Enumeration areas are usually *built* to be roughly equal in size.

1.26 Recovering both probabilities

```
cl["pi_rel"] = 1 / cl["w"]                          # overall, relative
cl["stage2"] = cl["take"] / cl["M_rel"]             # within-cluster, relative
cl["stage1"] = cl["pi_rel"] / cl["stage2"]          # what is left over

print("stage-1 probability across the 50 clusters:")
print("  cv = %.2e  (SRS says it should be constant)" % (cl["stage1"].std()
```

Zero to numerical precision. This is about the simplest case of two-stage cluster sampling:

1. EAs drawn with equal probability ($r_c = 1/\text{Number of EAs}$)
2. Households within a EA drawn with equal probability ($p_i = 1/M_{c_i}$)

1.27 Hazards of ignoring sample weights

Consider the following **Population Pyramid**, a nice graphical device for thinking about the demographic structure of a population. Because we're after population statistics, here we use the sample weights:

```
from lsms_library.visualizations import population_pyramid

ax = population_pyramid(glss, wave=WAVE, weights=True)
ax.figure.set_size_inches(7, 5.5)
```

1.28 Hazards of ignoring sample weights

Now, contrast if we *neglect* the survey weights, and just treat our data as if it was a simple random sample:

```
ax = population_pyramid(glss, wave=WAVE, weights=True, ghost=False)
ax.figure.set_size_inches(7, 5.5)
```

The filled bars are weighted; the outline is unweighted. The outline stands proud of the fill at every young age band and hugs it at the old ones, so the unweighted sample contains proportionally **more children** than the population it represents.

1.29 Why?

Let's consider two facts.

```
r = glss.household_roster(waves=[WAVE]).reset_index()
s = glss.sample(waves=[WAVE]).reset_index()

people = len(r)
weighted_total = s.set_index("i")["weight"].reindex(r["i"]).sum()
print(f"people in the roster ______: {people:,}")
print(f"sum of their weights ______: {weighted_total:,.1f}")
print(f"mean weight per household : {s['weight'].mean():.4f}")
```

The mean weight is 1 by construction. But that mean is over **households** while the roster is one row per **person**. So the weighted person-total is $\sum_h n_h w_h$, which equals the headcount only if weight and household size are uncorrelated.

So are they uncorrelated?

```
size = r.groupby("i").size().rename("hhsize")
d = s.set_index("i").join(size).dropna(subset=["hhsize"])
print("corr(weight, household size) =", round(d["weight"].corr(d["hhsize"]),
print()
print(d.groupby(pd.cut(d.hhsize, [0, 1, 2, 3, 5, 8, 50],
                       labels=["1", "2", "3", "4–5", "6–8", "9+"]),
                observed=True)
      .agg(households=("weight", "size"), mean_weight=("weight", "mean"))
      .round(3).to_string())
```

Bigger households carry smaller weights. Since bigger households hold more children, an unweighted count over-represents the young. But household size is a **symptom**: nothing in a sample design weights on household size directly. The question is what the design was actually doing.

2 Describing Welfare

Constructing a consumption aggregate, and then describing the distribution of it, before modelling anything.

2.1 Reading

Deaton is in your `reading/` folder on the hub, or PDF.

- Deaton, chs. 3 and 4
- Deaton & Zaidi, *Guidelines for Constructing Consumption Aggregates* — the operational reference, and the one you will actually reach for

- Deaton & Zaidi §§2–4 if you read nothing else

2.2 Why consumption, not income

- Income in an agricultural economy is *seasonal*, *lumpy*, and badly measured
  - "how much did you earn last year?" from a farmer with three crops
- Consumption is smoother, and closer to the theoretical object (permanent income, lifetime resources)
- Consumption is also easier to ask about: recall over short windows, item by item
- The cost: constructing it is real work. That work is this session.

2.3 Why consumption, not income

The argument for consumption is often stated as though it were about measurement error alone, and it is not. Under the permanent-income hypothesis, consumption is the household's own summary statistic for lifetime resources — it is what the household believes about its own position, revealed by its behaviour. Income in a single year is a noisy draw around that. So the case for consumption is partly statistical (smoother, better measured) and partly theoretical (closer to the welfare concept), and the two arguments reinforce each other.

The case is weakest where credit and insurance are good, since then consumption is smooth *because* households are smoothing, and the smoothness you are exploiting has destroyed the variation you wanted. That is not the situation in most LSMS settings, which is why the practice is near-universal there and rarer in the OECD.

2.4 The four components

A consumption aggregate is a sum of four things, in decreasing order of how well they are measured:

1. **Food** — purchased, own-produced, and received as gifts
2. **Non-food, non-durable** — fuel, transport, clothing, school fees, health

3. **Durables** — *use value*, not purchase price
4. **Housing** — rent, actual or imputed

Each of the last three involves an imputation, and each imputation is a place where two careful analysts will differ.

2.5 Own production and the valuation problem

Much of the food in an LSMS household was never bought.

- Value at what price?
  - farmgate price received (what the household gave up)
  - local market price (what it would have cost)
  - these differ by the marketing margin, which can be 30–50%
- Deaton & Zaidi: value at the price the household *faces*, consistently across households
- In practice: build a median unit value by item, by cluster, by round, and fall back up the hierarchy when cells are thin

2.6 Own production and the valuation problem

The unit-value hierarchy is worth dwelling on, because it is the first place in the workshop where a defensible answer requires a judgment rather than a formula. You have, for a given item, a set of reported (expenditure, quantity) pairs from households that bought it. Their ratio is a unit value, not a price: it embeds quality choice, package size, and reporting error, and a rich household's unit value for "rice" is higher partly because they are buying better rice.

The standard move is to take the median within the smallest cell with enough observations — cluster, then district, then region, then national — and use it for the households that did not buy. The median rather than the mean because unit values have a long right tail from misreported quantities. The hierarchy because the cluster cell will often be empty. And you should report how far up the hierarchy you had to climb, since an aggregate built mostly on national medians is a different object from one built on cluster medians.

2.7 Durables and housing

Durables: what enters consumption is the *flow of services*, not the purchase. For a stock S with service life L and interest rate r,

$$\text{use value} \;\approx\; S\left(r + \frac{1}{L}\right).$$

Housing: for owner-occupiers there is no rent, so impute it.

- hedonic regression of log rent on housing characteristics, among renters
- predict for owners
- fragile where the rental market is thin, which is most rural areas

2.8 Deflation

Two deflations, and they are different problems:

- **Temporal**: prices rise between rounds. Use the CPI, or better, a survey-based Paasche index built from the survey's own unit values.
- **Spatial**: prices differ across regions *within* a round, often by more than they differ across a decade. A national poverty line applied to undeflated nominal consumption will find poverty wherever prices are high.

Deaton & Zaidi recommend a household-specific Paasche index

$$P_i = \left[\sum_j w_{ij}\left(\frac{p_{ij}}{p_j^0}\right)^{-1}\right]^{-1},$$

with w_{ij} household i's budget share on item j.

2.9 Deflation

Spatial deflation is the step most often skipped and most often decisive. In a country the size of Ghana, the price of a fixed food basket can differ by 30% between Accra and the Upper East, which is comparable to the entire national increase in real consumption over a decade. An analysis that deflates over time but not over space will conclude that the north is poor for reasons that are partly an artefact of the accounting.

It also matters that the index is *household-specific*. A regional price index applies the same deflator to everyone in the region, and so implicitly assumes they consume the same bundle. They do not: the poor spend more of their budget on staples, so the relevant price index for the poor moves with staple prices. This is not a refinement; it is what makes the poverty rate respond correctly to a maize price shock.

2.10 Adult equivalence

A household of six is not six times as well off as a household of one with the same total consumption. Two adjustments, usually conflated:

- **Household composition**: children cost less than adults — ⇒ adult-equivalent scale $A_i = \sum_k a_{\text{age,sex}}$
- **Economies of scale**: shared goods, so cost rises less than proportionately — ⇒ n_i^{θ}, with $\theta \in (0, 1)$

Combined: $c_i = \dfrac{C_i}{A_i^{\theta}}$.

θ is not estimated in this session; it is *assumed*. Report the poverty rate for several values of it.

2.11 Poverty measures

The Foster–Greer–Thorbecke class: for poverty line z,

$$P_\alpha = \frac{1}{N} \sum_{i:c_i<z} \left(\frac{z - c_i}{z} \right)^{\alpha}.$$

- $\alpha = 0$: headcount ratio. Ignores *how* poor the poor are.
- $\alpha = 1$: poverty gap. The transfer needed, per capita, as a share of z.
- $\alpha = 2$: squared gap. Weights the poorest most; the only member of the family that satisfies the transfer axiom.

Weighted, always: $P_\alpha = \sum_i w_i \left(\frac{z-c_i}{z}\right)^{\alpha} \Big/ \sum_i w_i$.

2.12 Inequality

- **Lorenz curve**: cumulative share of consumption against cumulative share of population. Everything else is a summary of it.
- **Gini**: twice the area between the Lorenz curve and the 45° line. Bounded, familiar, and insensitive where it matters most.
- **Generalized entropy** $GE(\alpha)$: decomposable into within- and between-group parts. $GE(0)$ is mean log deviation, $GE(1)$ is Theil.

Prefer the whole Lorenz curve to any scalar. Prefer a decomposable scalar to the Gini when you want to say *where* inequality lives.

2.13 Building the aggregate

`LSMS_Library` does the food side for you. Start from what was acquired:

```
# Show tracebacks without the library's internal frames: the line that
# failed, and why.  Change Plain to Verbose if you ever want the rest.
%xmode Plain

import lsms_library as ll
import numpy as np, pandas as pd

ghana = ll.Country('GhanaLSS')
acq = ghana.food_acquired()
acq.head()
```

The index is $(t, i, j, u, s, visit)$ — wave, household, item, unit, source, visit — and the columns are `Expenditure`, `Quantity`, `Price`. The derived table collapses this to expenditure per household-item:

```
food = ghana.food_expenditures()
food.head()
```

Total food consumption per household is then a sum over items:

```
food_total = food.groupby(['t', 'i']).sum().squeeze()
food_total.groupby('t').describe().round(0)
```

2.14 Unit values, and what they hide

Look at the unit values directly, for one item, in one round.

```
uv = (acq.Expenditure / acq.Quantity).rename('unit_value')
uv = uv.replace([np.inf, -np.inf], np.nan).dropna()

item = uv.xs('2016-17', level='t').groupby('j').size().idxmax()
# commonest item
x = uv.xs('2016-17', level='t').xs(item, level='j')
print(item, ':', len(x), 'observations')
x.groupby('u').describe().round(2)            # by unit of measure
```

Two lessons, both visible in that table. Unit values vary enormously within an item — ratios of 100 to 1 are common, and are misreported quantities, not real price dispersion. And they are only comparable *within* a unit of measure, which is why `u` is in the index.

```
# The median is the robust summary; the mean is not.
x.groupby('u').agg(['median', 'mean', 'count']).round(2).head()
```

2.15 Poverty, correctly weighted

```
sample = ghana.sample()

def fgt(c, w, z, alpha=0):
    """Weighted FGT_alpha.  c: consumption per adult equivalent; w: weights
    c, w = np.asarray(c, float), np.asarray(w, float)
    ok = np.isfinite(c) & np.isfinite(w)
    c, w = c[ok], w[ok]
    # Sum over the poor only.  Writing this as np.sum(w * gap**alpha) over
    # everybody looks equivalent, and is — except at alpha=0, where
    # 0**0 == 1 and every household in the country counts as poor.
    poor = c < z
    return np.sum(w[poor] * ((z - c[poor]) / z) ** alpha) / np.sum(w)

wave = '2016-17'
c = food_total.xs(wave, level='t')
w = sample.xs(wave, level='t').weight.reindex(c.index)

z = c.quantile(0.25)            # a placeholder line; see the exercise
for a in (0, 1, 2):
    print(f"P_{a} = {fgt(c, w, z, a):.4f}")
```

```
# The line is an unweighted quartile, so the UNWEIGHTED headcount must come
# back as 0.25 by construction.  It is the one number here we know in
# advance, which makes it the right thing to check the code against.
print(f"unweighted P_0 = {fgt(c, np.ones(len(c)), z, 0):.4f}")
print(f"mean weight, poor = {w[c < z].mean():.3f}"
      f"   non-poor = {w[c >= z].mean():.3f}")
```

The unweighted headcount returns 0.2498, which is 0.25 up to the discreteness of the sample — as it must be, since the line *is* the unweighted lower quartile. That is the check: it is the one quantity here whose value you know before running the code, so it is the one that tells you whether the code is right. If it comes back as 1.0000, you have found the 0^0 trap.

The weighted headcount is 0.1537 — nine percentage points lower. The reason is in the line below it: poor households carry a mean weight of 0.62 against 1.13 for everyone else, because the design over-sampled them. Session 1's lesson, arriving as a number you might otherwise have published.

None of which rescues the line itself. A quartile of the distribution is not a poverty line: it fixes the unweighted headcount at 0.25 by construction, in every country and every year, which is a fact about arithmetic rather than about Ghana. A real line is absolute — the cost of a fixed bundle — and does not move with the distribution, and does not move when everyone gets poorer. Fixing that is the first exercise.

2.16 Lorenz and Gini

```
def lorenz(c, w):
    c, w = np.asarray(c, float), np.asarray(w, float)
    ok = np.isfinite(c) & np.isfinite(w) & (c >= 0)
    c, w = c[ok], w[ok]
    order = np.argsort(c)
    c, w = c[order], w[order]
    p = np.cumsum(w) / np.sum(w)
    L = np.cumsum(w * c) / np.sum(w * c)
    return np.concatenate([[0], p]), np.concatenate([[0], L])

def gini(c, w):
    p, L = lorenz(c, w)
    return 1 - 2 * np.trapezoid(L, p)

print(f"Gini (food consumption, {wave}) = {gini(c, w):.3f}")
```

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(5, 5))
for wv in ['1998-99', '2005-06', '2012-13', '2016-17']:
    cc = food_total.xs(wv, level='t')
    ww = sample.xs(wv, level='t').weight.reindex(cc.index)
    p, L = lorenz(cc, ww)
    ax.plot(p, L, label=f"{wv}  (G={gini(cc, ww):.2f})")
ax.plot([0, 1], [0, 1], 'k--', lw=0.8)
ax.set_xlabel('cumulative share of households')
ax.set_ylabel('cumulative share of food consumption')
ax.legend(frameon=False)
plt.show()
```

If two Lorenz curves cross, the ranking of the two distributions depends on which inequality measure you chose, and no scalar will rescue you. Look before you summarize.

2.17 One index, or several?

Everything in this session divides nominal expenditure by a scalar: a Paasche index over space, a CPI over time, an equivalence scale over composition. Three scalars, one household, one number out.

That procedure is exactly right if the demand system has **rank one**.

- rank one means budget shares do not vary with total expenditure
- but this session opened by showing that they do — that is Engel's law
- so on what grounds is a single deflator adequate?

Hold the question. Session 3 estimates the rank for Uganda and answers it, and the answer is not reassuring.

2.18 One index, or several?

It would be dishonest to teach the Deaton–Zaidi procedure as best practice and leave it there, because the profession's own best estimates say the procedure is not adequate for the data we are using it on. It is also the procedure every statistical agency uses, it is what your referees will expect, and you must be able to execute it competently — which is why it comes first and in full.

The order is deliberate: learn the standard method properly here, learn in session 3 what it costs, and then decide, case by case, whether the cost matters for the question in front of you. That is a different thing from never having known.

2.19 Exercises

1. The poverty line used above is fake. Build a real one: take the food bundle consumed by households in the second and third deciles, price it at national median unit values, and scale it to 2,900 kcal per adult equivalent. Recompute P_0, P_1, P_2.

2. Deflate spatially. Construct a regional Paasche index from the survey's own unit values, apply it, and report how much of the north–south poverty gradient survives.

3. Vary θ in $c_i = C_i / A_i^{\theta}$ over $[0.5, 1.0]$ and plot the headcount ratio against it. Over what range does the *ranking* of regions change?

3 Demand and Welfare

Where household surveys earn their keep for policy: what happens to whom when a price moves.

3.1 Reading

Deaton is in your `reading/` folder on the hub, or PDF.

- Deaton, chs. 4 (§4.2 on Engel curves) and 5 (all)
- Deaton & Muellbauer (1980), "An Almost Ideal Demand System," *AER* 70:312–326
- Lewbel (1991), "The rank of demand systems," *Econometrica* 59:711–730
- Ligon, "Household Welfare from Disaggregate Demands" (working paper) — the CFE system we estimate today

3.2 Budget shares and Engel curves

Let $w_{ij} = p_j q_{ij}/x_i$ be household i's budget share on good j, x_i total expenditure. The Working–Leser form,

$$w_{ij} = \alpha_j + \beta_j \log x_i + \gamma_j' z_i + u_{ij},$$

fits food shares remarkably well across countries and decades.

- $\beta_{\text{food}} < 0$ is Engel's law
- z_i: household size and composition, region, season
- Estimated by OLS, weighted, clustered on the EA

3.3 Engel's law as a diagnostic

The food share is the oldest welfare indicator in economics and still one of the better ones, precisely because it requires no price data and no deflation. If your constructed consumption aggregate does not produce a declining food share in log total expenditure, something is wrong with the aggregate, and you should find out what before going further. It is the cheapest specification check available, and it catches a surprising share of the errors that arise in session 2 — a mis-scaled own-production imputation, in particular, shows up immediately as a food share that is flat or rising.

3.4 Equivalence scales from behaviour

Session 2 *assumed* a scale. Demand analysis lets you estimate one.

- **Engel method**: two households are equally well off if they have the same food share. Scale = the expenditure ratio that equalizes it.
- **Rothbarth method**: use expenditure on adult goods (alcohol, tobacco, adult clothing) as the indicator instead.
- The two disagree, systematically and by a lot. Deaton ch. 4 explains why: the Engel method conflates the cost of a child with the child's effect on the household's needs for food specifically.

3.5 Prices, and where to get them

LSMS surveys rarely carry a price questionnaire. They carry unit values, and unit values are contaminated by quality choice:

$$\log v_{ij} = \log p_j + \underbrace{\psi_j \log x_i}_{\text{quality}} + \epsilon_{ij}.$$

Deaton's solution exploits the design: *within a cluster*, all households face the same price. So

- regress $\log v_{ij}$ on $\log x_i$ and household characteristics *with cluster fixed effects* to strip out quality;
- the cluster effects are the price variation you wanted.

This is the single best argument for caring about `v` in the index.

3.6 The rank of a demand system

Stack the Engel curves. The *rank* of a demand system (Lewbel 1991) is the dimension of the space spanned by the budget shares viewed as functions of total expenditure.

- rank 1: shares do not vary with x — homothetic
- rank 2: Working–Leser, AIDS
- rank 3: QUAIDS; Gorman's bound for exactly aggregable systems

Why you should care: **if the rank exceeds one, no single price index can deflate nominal expenditure so as to keep it monotone in utility when relative prices move.** Rank r needs r indices.

3.7 The rank of a demand system

This is the result that indicts session 2, and it is worth being precise about the indictment. Everything we did there — the Deaton–Zaidi household-specific Paasche index, the spatial deflator, the temporal one — constructs a *single* scalar per household and divides by it. That procedure is exactly right if the demand system has rank one, defensible as an approximation at rank two, and simply not available at higher rank: there is no scalar function of prices whose quotient with nominal expenditure orders households the way utility does.

Estimates for Uganda put the rank at three at least, with a best estimate of six. So the deflation in session 2 is not a refinement we skipped for lack of time; it is a procedure that cannot be made correct by being more careful with the index. What follows is the alternative: estimate the welfare number directly from behaviour, and never construct a price index at all.

3.8 Constant Frisch Elasticity demands

Let λ_i be household i's marginal utility of expenditure. Frischian (λ-constant) demands take the CFE form

$$f_j(p,\lambda) = \left(\frac{\alpha_j}{\lambda p_j}\right)^{\beta_j}, \qquad U(c) = \sum_j \alpha_j \beta_j \frac{c_j^{1-1/\beta_j} - 1}{\beta_j - 1}.$$

- β_j is the **Frisch elasticity** of demand for good j: necessities small, luxuries large
- $\beta_j = \beta$ for all j gives CES; $\beta \to 1$ gives Cobb–Douglas
- unrestricted rank, so it does not impose what we just said is false

3.9 The estimating equation

Take logs of expenditure on good j. Writing $w_i \equiv -\log \lambda_i$,

$$\log x_{it}^j \;=\; a_t^j \;+\; \beta_j\, w_{it} \;+\; \gamma_j' d_{it} \;+\; \varepsilon_{it}^j.$$

This is a **one-factor model**. The $(it) \times j$ matrix of log expenditures, once good-time effects a_t^j and characteristics d_{it} are swept out, is rank one: loadings β, factor w.

- estimate by SVD of the residual matrix, with missing data
- d_{it} enters as good-specific Barten scales, not "independence of base"
- needs **no prices** and **no total expenditure** — only expenditures on a subset of goods

3.10 $w = -\log\lambda$ as a welfare measure

λ is the marginal utility of expenditure, so it *falls* as a household gets better off. Hence $w = -\log\lambda$ is increasing in welfare.

- comparable across households facing different prices — the prices are in a_t^j, not in w
- comparable across household compositions — composition is in $\gamma_j' d$
- derived from behaviour, not from an assumed equivalence scale or an assumed price index
- w is the partial derivative of indirect utility with respect to total expenditure: a genuine welfare object, though not itself a utility level

3.11 $w = -\log \lambda$ as a welfare measure

It is worth being clear about what this does and does not deliver, because the claim is easy to overstate in both directions.

What it delivers: a scalar per household-period that is comparable across households, estimated from expenditures on a subset of goods, with no price data and no total-expenditure measure. That is a remarkable amount to get from very little, and it is precisely the situation an LSMS analyst is in.

What it does not deliver: a utility level. One cannot in general integrate back from λ to the indirect utility function — that would require demands for *all* goods. So w supports statements about who is better off than whom, and about how a distribution shifts; it does not support a claim to have measured utility. Much of the appeal is exactly that the weaker claim is enough for the questions we actually ask of these data.

3.12 Welfare consequences of a price change

For a small change in the price of good j, the compensating variation is approximately

$$\mathrm{CV}_i \;\approx\; q_{ij}\,\Delta p_j \;=\; w_{ij} x_i \frac{\Delta p_j}{p_j}.$$

- to first order you need only the *budget share* — no elasticities
- the distributional incidence follows from how w_{ij} varies with x_i, which is the Engel curve you already estimated
- second-order terms need the demand system, and are usually small relative to the first-order term for modest price changes

3.13 Welfare consequences of a price change

This is the payoff for the whole exercise, and it is worth being explicit about how little machinery the first-order answer requires. A finance ministry contemplating the removal of a fuel subsidy does not need a structural demand system to know the first-order incidence: it needs the budget share of fuel by decile, which is one `groupby` on a well-constructed consumption aggregate. The demand system buys you the behavioural response — how much households substitute away — which matters for the revenue calculation and for large changes, and much less for the distributional one.

The order of operations in policy work therefore usually runs backwards from the way it is taught. Get the aggregate right, get the shares right, report the first-order incidence with standard errors, and only then argue about elasticities.

3.14 Budget shares

```
# Show tracebacks without the library's internal frames: the line that
# failed, and why.  Change Plain to Verbose if you ever want the rest.
%xmode Plain

import lsms_library as ll
import numpy as np, pandas as pd

ghana = ll.Country('GhanaLSS')
food = ghana.food_expenditures()
sample = ghana.sample()

wave = '2016-17'
# food_expenditures is indexed (i, t, v, j, s).  Collapse to one row per
# household-item before doing anything else: unstacking the raw index
# would ask pandas for a 13918 x 373678 array.
f = food.xs(wave, level='t').squeeze().groupby(['i', 'j']).sum()
x = f.groupby('i').sum()                          # total food expenditure
shares = f.div(x, level='i').unstack('j').fillna(0.0)
shares.shape

# The ten largest items by mean budget share
top = shares.mean().sort_values(ascending=False).head(10)
top.round(4)
```

3.15 An Engel curve

```
import statsmodels.api as sm

# household_characteristics is a count of people by age-sex cell, indexed
# (t, v, i).  Sum across cells for household size, then drop to i alone so
# it aligns with the shares.
chars = ghana.household_characteristics().xs(wave, level='t')
hhsize = chars.sum(axis=1).groupby('i').first().rename('hhsize')

s1 = sample.xs(wave, level='t')
item = top.index[0]

d = pd.concat([shares[item].rename('w'), x.rename('x'), hhsize,
               s1.v.rename('v'), s1.weight.rename('wt')], axis=1).dropna()
d = d[(d.x > 0) & (d.hhsize > 0)]
d['logx'] = np.log(d.x.astype(float))
d['logn'] = np.log(d.hhsize.astype(float))

res = sm.WLS(d.w.astype(float),
             sm.add_constant(d[['logx', 'logn']].astype(float)),
             weights=d.wt.astype(float)
             ).fit(cov_type='cluster', cov_kwds={'groups': d.v.astype(str)})
print(res.summary().tables[1])
```

Note `cov_type'cluster'. Session 1 established why: without it the standard errors on =logx` are too small by roughly $\sqrt{\text{deff}}$.

```
import matplotlib.pyplot as plt

# Nonparametric Engel curve: weighted mean share by expenditure ventile.
# A weighted mean is a ratio of two sums, so no groupby-apply is needed.
d['bin'] = pd.qcut(d.logx, 20, labels=False, duplicates='drop')
curve = (d.w * d.wt).groupby(d.bin).sum() / d.wt.groupby(d.bin).sum()
mid = d.logx.groupby(d.bin).mean()

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(mid, curve, 'o-')
ax.set_xlabel(r'$\log x$   (total food expenditure)')
ax.set_ylabel(f'budget share, {item}')
plt.show()
```

3.16 Stripping quality out of unit values

```
acq = ghana.food_acquired().xs(wave, level='t')
uv = np.log((acq.Expenditure / acq.Quantity).replace([np.inf, -np.inf], np.

j0 = uv.groupby('j').size().idxmax()
u0 = uv.xs(j0, level='j').groupby('i').mean().rename('logv')

e = pd.concat([u0, d.logx, d.v], axis=1).dropna()

# Deaton's within-cluster estimator.  There are ~1000 clusters, so do not
# build the dummy matrix; by Frisch-Waugh-Lovell, demeaning within cluster
# gives the identical coefficient at a fraction of the cost.
for col in ('logv', 'logx'):
    e[col + '_d'] = e[col].astype(float) - e.groupby('v')[col].transform('m

within = sm.OLS(e.logv_d, e[['logx_d']]).fit(
    cov_type='cluster', cov_kwds={'groups': e.v.astype(str)})
print(f"quality elasticity psi = {within.params['logx_d']:.3f}"
      f"    (se {within.bse['logx_d']:.3f})")
```

A ψ near zero says households of different means pay the same for this item — it is a homogeneous good. A large ψ says "rice" in the questionnaire is several different goods in the market.

3.17 Estimating a CFE system

The CFE estimator needs several waves and a food classification that is consistent across them. Ghana's GLSS has the waves but no harmonized food aggregation; Uganda has both, and it is the panel we use for the rest of the workshop. So we switch countries here, deliberately.

```
import lsms_library as ll
from cfe import Regression
import numpy as np, pandas as pd

uga = ll.Country('Uganda')
x = uga.food_expenditures().squeeze()

# Collapse the survey's fine item codes onto the harmonized aggregate
# labels: 'Matoke (bunch)', 'Matoke (heap)', ... all become 'Matoke'.
```

```
agg = (uga.categorical_mapping['harmonize_food']
           .set_index('Preferred Label')['Aggregate Label'].to_dict())
x = x.rename(index=agg, level='j')
x = x.groupby(x.index.names).sum()
print(x.index.get_level_values('j').nunique(), 'goods after aggregation')
```

`cfe` insists on the index being exactly (i, t, m, j) — household, period, *market*, good. A market is a set of households facing common prices, so the enumeration area is the natural choice; we start with a single market, which puts all the price variation into the good-time effects a_t^j.

```
y = np.log(x.replace(0, np.nan).dropna()).groupby(['i', 't', 'j']).sum()
y = pd.concat({1: y}, names=['m']).reorder_levels(['i', 't', 'm', 'j']).sor

d0 = uga.household_characteristics()
d = d0.assign(
    Girls=d0[[f'F {a}' for a in ['00-03', '04-08', '09-13', '14-18']]].sum(
    Boys =d0[[f'M {a}' for a in ['00-03', '04-08', '09-13', '14-18']]].sum(
    Women=d0[[f'F {a}' for a in ['19-30', '31-50', '51+']]].sum(axis=1),
    Men  =d0[[f'M {a}' for a in ['19-30', '31-50', '51+']]].sum(axis=1),
)[['Girls', 'Boys', 'Women', 'Men', 'log HSize']].dropna(how='any')
d = d.groupby(['i', 't']).first()
d = pd.concat({1: d}, names=['m']).reorder_levels(['i', 't', 'm']).sort_ind

r = Regression(y=y, d=d)
beta = r.get_beta()            # a few seconds
print(len(beta), 'goods estimated')
```

3.18 Frisch elasticities

```
print("least elastic:"); print(beta.sort_values().head(6).round(2).to_strin
print("\nmost elastic:"); print(beta.sort_values().tail(6).round(2).to_stri
```

Read those two lists before reading anything else. The ordering is the model's main testable implication and it is not imposed: nothing in the estimator knows which goods are staples. If salt and cassava do not come out at the bottom and fruit and coffee at the top, something is wrong with the data or with the aggregation.

```
ax = r.graph_beta(xlabel=r'Estimates of $\beta$')
```

We did not estimate demands for all seventy-six goods. Items too few households ever report cannot support a coefficient, and the estimator drops

them without being asked. Report how many survived, because the answer depends on the aggregation you chose.

Household composition enters through γ, one coefficient per good per characteristic — good-specific Barten scales rather than a single equivalence scale imposed on everything:

```
r.get_gamma().round(2).head(8)
```

Before trusting any of it, look at the fit:

```
import matplotlib.pyplot as plt

fit = pd.DataFrame({'actual': r.y,
                    'predicted': r.get_predicted_log_expenditures()}).dropn
ax = fit.plot.scatter(x='predicted', y='actual', s=1, alpha=0.15, figsize=(
lim = [fit.min().min(), fit.max().max()]
ax.plot(lim, lim, 'k—', lw=0.8)
ax.set_title(f"log expenditures, R^2 = {fit.corr().iloc[0, 1] ** 2:.2f}")
plt.show()
```

Now the welfare measure itself. This one takes about half a minute.

```
w = r.get_w()                    # w = -log lambda, indexed (i, t, m)
w.groupby('t').describe()[['count', 'mean', 'std']].round(3)
```

Estimating this took most of a minute, and sessions 4 and 5 need the same object. Save it rather than refitting: the pickle carries β, γ, w, and the predicted expenditures together.

```
from pathlib import Path

r.predicted_expenditures()              # populate before saving
out = Path.home() / '.cache' / 'hhsurveys' / 'uganda.rgsn'
out.parent.mkdir(parents=True, exist_ok=True)
r.to_pickle(str(out))

# and to get it back:
#   import cfe
#   r = cfe.read_pickle(str(out))
print('saved', out)
```

3.19 The Engel pie

An Engel curve shows one good at a time. With a whole system estimated we can show all of them at once: draw the predicted budget *composition* as

a pie, and let the radius be $\log x$. Poor households at the centre, rich at the rim.

```
import matplotlib.pyplot as plt
from matplotlib import cm

xhat = r.predicted_expenditures()
xbar = xhat.groupby(['i', 't', 'm']).sum()
p_j = ((r.y.unstack('j') > 0) + 0.).mean()        # prob. good j is purchased

lo, hi = xbar.quantile(0.05), xbar.quantile(0.95)
Y = np.geomspace(lo, hi, 60)

fig, ax = plt.subplots(figsize=(7.5, 6))
wedges = None
for k in range(len(Y) - 1, 0, -1):
    ax.set_prop_cycle('color', cm.tab20.colors)
    shares = r.expenditures(Y[k]) * p_j
    out = ax.pie(shares, radius=np.log(Y[k]) / np.log(hi), counterclock=Fal
    if wedges is None:                            # keep the outermost ring
        wedges, goods = out[0], shares.index.tolist()

ax.arrow(0, 0, 1, 0, shape='full', head_width=.04, length_includes_head=Tru
ax.annotate(r'$\log x$', xy=(1, 0), color='red', va='center_baseline')

# 41 goods will not fit in a legend; name the ten largest at the rim.
big = np.argsort(r.expenditures(Y[-1]).to_numpy() * p_j.to_numpy())[::-1][:
ax.legend([wedges[b] for b in big], [goods[b] for b in big],
          loc='center left', bbox_to_anchor=(1.02, 0.5),
          frameon=False, fontsize=8, title='largest ten, at the rim')
plt.show()
```

Wedges that widen as you move outward are luxuries; wedges that pinch are necessities. This is Engel's law, for forty-one goods simultaneously, and it is the same information as the β plot in a form you can show to someone who does not know what a Frisch elasticity is.

3.20 Exercises

1. Estimate the Working–Leser food share equation on the *full* consumption aggregate rather than food alone, and test whether β differs be-

tween urban and rural households.

2. Implement the Engel equivalence scale: find the expenditure ratio that equates the predicted food share of a household with two adults and two children to that of a household with two adults. Compare with the θ-scale you used in session 2.

3. Compute the first-order welfare cost of a 20% rise in the price of the largest food item, by consumption decile. Who loses most, in cedis and as a share of expenditure? Are those the same households?

4. Re-estimate the CFE system using the enumeration area as the market m rather than a single national market. How much do the β_j move? What have you assumed away by using one market?

5. Regress w on log total food expenditure. The two are measuring the same thing in principle; how closely do they agree, and which households do they disagree about?

4 From Cross-Sections to Dynamics

Two ways to get change out of survey data: build a panel from cohorts, or use one that already exists.

4.1 Reading

Deaton is in your `reading/` folder on the hub, or PDF.

- Deaton (1985), "Panel data from time series of cross-sections"
- Deaton, ch. 2 §2.7 and ch. 6
- Abadie, Athey, Imbens & Wooldridge (2023), "When should you adjust standard errors for clustering?" *QJE* 138:1–35

4.2 The problem

Ghana's GLSS gives seven rounds, but almost never the same household twice. We want

$$\Delta \log c_{it} = \alpha_i + \delta_t + \beta' \Delta z_{it} + \epsilon_{it},$$

and α_i is not identified because i appears once.

Deaton's answer: give up on households and follow *cohorts*. A cohort — say, everyone born 1965–69 — is present in every round, and its membership is fixed by construction.

4.3 Pseudo-panels

Define cohort c by birth year of the head. For each (c, t) compute the weighted mean $\bar{y}_{ct}$. Then

$$\bar{y}_{ct} = \alpha_c + \delta_t + \beta' \bar{z}_{ct} + \bar{\epsilon}_{ct}$$

is a genuine panel, in $C \times T$ cells, and α_c is estimable.

The catch: $\bar{y}_{ct}$ is a *sample* mean, so it carries measurement error of order $1/\sqrt{n_{ct}}$. With $\bar{z}$ on the right-hand side that is classical errors-in-variables, and $\hat{\beta}$ attenuates.

4.4 Cohort size is the design decision

Everything about a pseudo-panel turns on one tradeoff, and it is worth stating it as a tradeoff rather than as a rule of thumb. Narrow cohorts (five birth years) give you many cells, so many degrees of freedom, but few households per cell, so noisy cell means and severe attenuation. Wide cohorts (fifteen birth years) give precise cell means but few cells and a cohort that is internally heterogeneous, so the fixed effect α_c is absorbing variation you might have wanted.

Deaton's own guidance is that cells want at least a hundred observations, and Verbeek and Nijman later showed that the errors-in-variables correction is straightforward once you have the within-cell variances, which the survey gives you for free. The practical advice is: choose the cohort width, report the cell counts, and show that the answer does not change much at the next width up. An analysis that does not report n_{ct} is hiding the one number a reader needs.

4.5 Identification: age, cohort, and time

You cannot have all three. Since

$$\text{age} = \text{year} - \text{birth year},$$

age, period, and cohort effects are exactly collinear. Something must be restricted:

- drop period effects (a "pure lifecycle" reading), or
- restrict period effects to sum to zero and be orthogonal to trend (Deaton & Paxson's normalization), or
- bring in outside information on one of the three

This is not a technical nuisance to be worked around. It is a statement that the data cannot answer the question without an assumption, and the assumption should be argued for.

4.6 A genuine panel: Uganda

The Uganda National Panel Survey follows households from 2005–06 to 2018–19. With the same i in multiple t:

$$y_{it} = \alpha_i + \delta_t + \beta' z_{it} + \epsilon_{it}$$

is estimable by within transformation.

- α_i absorbs everything time-invariant — soil quality, ability, the district's road
- what remains identified is the effect of variation *within* the household over time
- and β is now a within estimate, which is a different parameter from the cross-sectional one, not a better-identified version of it

4.7 Inference

Cluster at the level of treatment assignment, not at the level of sampling.

- sampling clusters (EAs): cluster the SEs, per session 1
- if the variation you exploit is at district level, cluster on district
- few clusters ($G < 40$) $\Rightarrow$ wild cluster bootstrap
- two-way (household and time) when T is not small

Abadie et al. (2023): clustering is about *design* or *sampling*, and if neither applies, clustering is not conservative — it is wrong.

4.8 Building a pseudo-panel

```
# Show tracebacks without the library's internal frames: the line that
# failed, and why.  Change Plain to Verbose if you ever want the rest.
%xmode Plain

import lsms_library as ll
import numpy as np, pandas as pd

ghana = ll.Country('GhanaLSS')
roster = ghana.household_roster()
sample = ghana.sample()
food = ghana.food_expenditures()

# Age of the head, by (t, i).  Relationship == 'Head' identifies the head.
heads = roster[roster.Relationship.str.strip().str.lower().eq('head')]
age = heads.groupby(['t', 'i']).Age.first().rename('age')
age.groupby('t').describe().round(1)

# Wave midpoint year, so we can turn age into a birth year
year = {w: int(w[:4]) + 1 for w in ghana.waves}

x = food.groupby(['t', 'i']).sum().squeeze().rename('c')
d = pd.concat([x, age], axis=1).dropna()
d['year'] = [year[t] for t in d.index.get_level_values('t')]
d['born'] = d.year - d.age
d = d[(d.age >= 25) & (d.age <= 70)]

# Five-year cohorts
d['cohort'] = (d.born // 5) * 5
d['logc'] = np.log(d.c.where(d.c > 0))

# `sample` is indexed (i, t); everything else here is (t, i).
reindex
# against a differently-ordered MultiIndex does not raise — it returns all
# NaN, and the pseudo-panel below comes out empty with no error anywhere.
# Reorder explicitly, then check that the merge actually matched.
w_all = sample.weight.reorder_levels(['t', 'i']).sort_index()
d['w'] = w_all.reindex(d.index)
assert d.w.notna().mean() > 0.9, f"only {d.w.notna().mean():.1%} of weights
```

```
# Weighted cell means, as ratios of sums.  groupby-apply returning a Series
# is fragile across pandas versions; this is not.
dd = d.dropna(subset=['logc', 'w']).reset_index()
grp = dd.groupby(['cohort', 't'])
cells = pd.DataFrame({
    'logc': (dd.w * dd.logc).groupby([dd.cohort, dd.t]).sum() / grp.w.sum()
    'age':  (dd.w * dd.age).groupby([dd.cohort, dd.t]).sum() / grp.w.sum(),
    'n':    grp.size(),
})
cells.head(12)

# Deaton's rule of thumb: cells want n >= 100.  How many survive?
print(f"{len(cells)} cells; {(cells.n >= 100).mean():.3f} of them have n >=
cells.n.describe().round(0)
```

4.9 Lifecycle consumption

```
import matplotlib.pyplot as plt

big = cells[cells.n >= 100].reset_index()
fig, ax = plt.subplots(figsize=(6.5, 4))
for c, g in big.groupby('cohort'):
    if len(g) >= 3:
        ax.plot(g.age, g.logc, 'o-', label=f"born {int(c)}-{int(c)+4}")
ax.set_xlabel('age of head')
ax.set_ylabel('mean log food consumption')
ax.legend(frameon=False, fontsize=7, ncol=2)
plt.show()
```

Each line is one cohort tracked across rounds. Where the lines lie on top of one another, there is no cohort effect and the shape is lifecycle. Where they are shifted vertically, later cohorts are better off at every age — a cohort effect. Where *every* line jumps in the same round, that is a period effect, or a change in the questionnaire.

4.10 The Uganda panel

```
uga = ll.Country('Uganda')
print(uga.waves)
print(uga.data_scheme)

import statsmodels.api as sm

ufood = uga.food_expenditures()
uc = ufood.groupby(['t', 'i']).sum().squeeze()
uc = np.log(uc.where(uc > 0)).rename('logc')

un = uga.household_characteristics().sum(axis=1).groupby(['t', 'i']).first(
un = np.log(un.astype(float).where(un > 0)).rename('logn')

p = pd.concat([uc, un], axis=1).dropna()
p = p[np.isfinite(p).all(axis=1)]

# How many households are actually observed more than once?
counts = p.groupby('i').size()
print(counts.value_counts().sort_index())

# Keep only households seen more than once -- singletons contribute nothing
# to a within estimator, and quietly inflate the apparent sample size.
p = p[counts.reindex(p.index.get_level_values('i')).to_numpy() > 1]

# Time dummies, then demean everything within household.
After demeaning
# there is no constant to include.
T = pd.get_dummies(p.index.get_level_values('t'), prefix='t',
                   drop_first=True, dtype=float)
T.index = p.index
q = pd.concat([p, T], axis=1)
qd = q - q.groupby(level='i').transform('mean')

res = sm.OLS(qd.logc, qd.drop(columns='logc')).fit(
    cov_type='cluster', cov_kwds={'groups': qd.index.get_level_values('i')}
print(res.summary().tables[1])
```

Compare the coefficient on `logn` here with the cross-sectional one from session 3. If they differ, the cross-sectional estimate was contaminated by whatever it is about large households that also predicts consumption.

4.11 The same panel, in welfare units

Everything above treats $\log c$ as the outcome. Session 3 argued that $w = -\log \lambda$ is the better welfare measure, because prices and household composition are swept into the good-time effects and the Barten scales rather than left in the number. Here is the same regression on w, so you can see how much it matters.

```
from pathlib import Path
import lsms_library as ll
import cfe
from cfe import Regression
import numpy as np, pandas as pd

def uganda_cfe():
    # The estimated CFE system for Uganda, as a cfe.Regression.
The object
    # carries beta, gamma, w = -log lambda, and predicted expenditures, so
    # nothing downstream has to refit.  Uses a copy staged on the hub if
    # there is one, else a copy you estimated earlier, else estimates it
    # from scratch (about forty seconds) and caches the result.
So this
    # cell is self-contained: no other notebook need have been run first.
    staged = Path('/srv/data/hhsurveys/uganda.rgsn')
    mine   = Path.home() / '.cache' / 'hhsurveys' / 'uganda.rgsn'
    for c in (staged, mine):
        if c.exists():
            return cfe.read_pickle(str(c))

    uga = ll.Country('Uganda')
    x = uga.food_expenditures().squeeze()
    agg = (uga.categorical_mapping['harmonize_food']
               .set_index('Preferred Label')['Aggregate Label'].to_dict())
    x = x.rename(index=agg, level='j')
    x = x.groupby(x.index.names).sum()

    y = np.log(x.replace(0, np.nan).dropna()).groupby(['i', 't', 'j']).sum(
    y = pd.concat({1: y}, names=['m']).reorder_levels(['i', 't', 'm', 'j'])

    d0 = uga.household_characteristics()
    d = d0.assign(
```

```
            Girls=d0[[f'F_{a}' for a in ['00-03', '04-08', '09-13', '14-18']]].
            Boys =d0[[f'M_{a}' for a in ['00-03', '04-08', '09-13', '14-18']]].
            Women=d0[[f'F_{a}' for a in ['19-30', '31-50', '51+']]].sum(axis=1)
            Men  =d0[[f'M_{a}' for a in ['19-30', '31-50', '51+']]].sum(axis=1)
        )[['Girls', 'Boys', 'Women', 'Men', 'log_HSize']].dropna(how='any')
        d = d.groupby(['i', 't']).first()
        d = pd.concat({1: d}, names=['m']).reorder_levels(['i', 't', 'm']).sort

        r = Regression(y=y, d=d)
        r.get_beta(); r.get_w(); r.predicted_expenditures()
        mine.parent.mkdir(parents=True, exist_ok=True)
        r.to_pickle(str(mine))
        return r

r = uganda_cfe()
w = r.get_w()
w.groupby('t').agg(['size', 'mean', 'std']).round(3)

# The same within estimator, in welfare units.
# w arrives indexed (i, t, m) while p is (t, i).  Line them up explicitly:
# concat on a differently-ordered MultiIndex does not raise, it just fails
# to align -- the same trap that emptied the pseudo-panel above.
wi = w.droplevel('m') if 'm' in (w.index.names or []) else w
wi = wi.rename('w').reorder_levels(['t', 'i']).sort_index()

q2 = pd.concat([wi, p.logn], axis=1).dropna()
assert len(q2) > 0.5 * min(len(wi), len(p)), "w and p failed to align"

c2 = q2.groupby('i').size()
q2 = q2[c2.reindex(q2.index.get_level_values('i')).to_numpy() > 1]

T2 = pd.get_dummies(q2.index.get_level_values('t'), prefix='t',
                    drop_first=True, dtype=float)
T2.index = q2.index
Q2 = pd.concat([q2, T2], axis=1)
Q2d = Q2 - Q2.groupby(level='i').transform('mean')

res_w = sm.OLS(Q2d.w, Q2d.drop(columns='w')).fit(
    cov_type='cluster', cov_kwds={'groups': Q2d.index.get_level_values('i')
print(res_w.summary().tables[1])
```

Three comparisons to make, and the third is the one to remember.

The coefficient on `logn`. In $\log c$ it is about $+0.34$: a bigger household simply spends more. In w it is about -0.03 — small and *negative*. Household size has already been absorbed into the Barten scales, so what is left is what size does to welfare net of need, and that is slightly adverse. The two numbers are not rival estimates of one parameter; they answer different questions.

The time effects. In $\log c$ they climb steeply and monotonically, from 0.40 to 1.20 across the panel. That is mostly inflation, since nothing deflated those expenditures. In w they sit within ± 0.15 and do not trend, because prices live in a_t^j and never entered w at all. No CPI was used to achieve that, and none was available.

Now look at 2009–10 in the w column. It is the only negative time effect in the panel. That is the 2008 food price crisis, and it is invisible in $\log c$, where 2009–10 is simply another step up the inflation ladder. Session 5 takes that observation and asks what it does to the poverty rate.

4.12 Exercises

1. Rebuild the Ghana pseudo-panel with ten-year cohorts. How much do the lifecycle profiles change? Which conclusion is robust?

2. Implement the Verbeek–Nijman errors-in-variables correction using the within-cell variances, and report the corrected β.

3. In Uganda, quantify attrition: what fraction of the 2009–10 households appear in 2011–12? Are the leavers different in 2009–10 consumption from the stayers? What does that do to your fixed-effects estimate?

5 Capstone and Frontiers

Where the methods break down, and then your own analysis.

5.1 Reading

5.1.1 Reading

- Ligon, "Household Welfare from Disaggregate Demands" (working paper)

- Ligon (2026), "Risk sharing tests and covariate shocks: Drought, Floods, and Pests in Uganda," *AER*, forthcoming

- Townsend (1994), "Risk and insurance in village India," *Econometrica*

5.2 Two measures of poverty, and they disagree

Take Uganda through the 2008 food price crisis, when cereal prices more than doubled. Anchor a headcount at 58% in 2005–06 and carry the *same* threshold forward.

- on $w = -\log\lambda$: poverty **rises** to about 63% by 2009–10
- on nominal per-capita expenditure: the median rises about 32% over the same interval

Deflate by anything less than 32% inflation and you conclude that welfare improved. The two measures disagree in **sign**, not in magnitude.

5.3 Two measures of poverty, and they disagree

This is the reason session 3 spent time on rank, and it is worth stating the mechanism rather than leaving it as a surprising number.

Deflating nominal expenditure by a price index answers the question "how much more of the 2005 bundle could this household buy?" When relative prices move — cereals doubling while other prices do not — that question stops tracking welfare, because the household does not want the 2005 bundle any more, and because the bundle it does want has become more expensive in a way no single scalar records. With rank at least three, three indices would be needed; with one you get an answer whose error is correlated with exactly the price movement you are trying to measure.

The λ estimate never forms an index. It reads welfare off the composition of expenditure directly, and the composition is precisely what responds to relative prices. That is why it can move in the opposite direction, and why, when the two disagree, the burden of proof is not on it.

5.4 Full risk sharing

If a group of households shares risk fully, they equate marginal utilities of expenditure up to fixed Pareto weights:

$$\lambda_{it} = \frac{\mu_t}{\theta_i} \quad \Longleftrightarrow \quad w_{it} = \log\theta_i - \log\mu_t.$$

So under full insurance w_{it} is a household effect plus a time effect, and **nothing else**. The test writes itself:

$$w_{it} = \alpha_i + \delta_t + \gamma' S_{it} + \varepsilon_{it},$$

with S_{it} household-level shocks. Full risk sharing implies $\gamma = 0$.

5.5 Why covariate shocks break the test

The test above has a hole, and it is exactly where the policy interest is.

- δ_t absorbs **everything** common to the group in period t
- a drought that hits the whole region *is* common to the group
- so the covariate component of the shock is differenced away by the very time effect that makes the test valid

Consequence: the standard test has power only against **idiosyncratic** shocks. Failing to reject tells you nothing about insurance against drought, flood, or pests — the risks that actually threaten a rural economy, and the ones a village cannot self-insure.

5.6 Why covariate shocks break the test

There is a general lesson here that outlives this application. A fixed effect is not free: it removes variation, and you have to ask what was in the variation it removed. Session 4 made the point in passing — household effects absorb time-invariant selection and nothing else — but here the cost is sharper, because the absorbed variation is not a nuisance. It is the treatment.

The way out is not econometric cleverness so much as data: identify who was exposed to the covariate shock and who was not, using something outside the survey's own time dimension — rainfall, remote-sensed vegetation, the spatial footprint of a flood — and then the exposure varies within a period and survives the time effect. That is what the paper does, and it is the reason it needs the LSMS-ISA plot data alongside the panel.

5.7 The sign flip, computed

```
# Show tracebacks without the library's internal frames: the line that
# failed, and why.  Change Plain to Verbose if you ever want the rest.
%xmode Plain
```

```
from pathlib import Path
import lsms_library as ll
import cfe
from cfe import Regression
import numpy as np, pandas as pd

def uganda_cfe():
    # The estimated CFE system for Uganda, as a cfe.Regression.
The object
    # carries beta, gamma, w = -log lambda, and predicted expenditures, so
    # nothing downstream has to refit.  Uses a copy staged on the hub if
    # there is one, else a copy you estimated earlier, else estimates it
    # from scratch (about forty seconds) and caches the result.
So this
    # cell is self-contained: no other notebook need have been run first.
    staged = Path('/srv/data/hhsurveys/uganda.rgsn')
    mine   = Path.home() / '.cache' / 'hhsurveys' / 'uganda.rgsn'
    for c in (staged, mine):
        if c.exists():
            return cfe.read_pickle(str(c))

    uga = ll.Country('Uganda')
    x = uga.food_expenditures().squeeze()
    agg = (uga.categorical_mapping['harmonize_food']
              .set_index('Preferred␣Label')['Aggregate␣Label'].to_dict())
    x = x.rename(index=agg, level='j')
    x = x.groupby(x.index.names).sum()

    y = np.log(x.replace(0, np.nan).dropna()).groupby(['i', 't', 'j']).sum(
    y = pd.concat({1: y}, names=['m']).reorder_levels(['i', 't', 'm', 'j'])

    d0 = uga.household_characteristics()
    d = d0.assign(
        Girls=d0[[f'F␣{a}' for a in ['00-03', '04-08', '09-13', '14-18']]].
        Boys =d0[[f'M␣{a}' for a in ['00-03', '04-08', '09-13', '14-18']]].
        Women=d0[[f'F␣{a}' for a in ['19-30', '31-50', '51+']]].sum(axis=1)
        Men  =d0[[f'M␣{a}' for a in ['19-30', '31-50', '51+']]].sum(axis=1)
    )[['Girls', 'Boys', 'Women', 'Men', 'log␣HSize']].dropna(how='any')
    d = d.groupby(['i', 't']).first()
```

```
    d = pd.concat({1: d}, names=['m']).reorder_levels(['i', 't', 'm']).sort

    r = Regression(y=y, d=d)
    r.get_beta(); r.get_w(); r.predicted_expenditures()
    mine.parent.mkdir(parents=True, exist_ok=True)
    r.to_pickle(str(mine))
    return r

r = uganda_cfe()
w = r.get_w()

waves = sorted(w.index.get_level_values('t').unique())
base = waves[0]

# Anchor at 58% in the base wave, then carry the SAME welfare threshold
# forward.  This is the whole exercise: one line, one threshold, no index.
z = w.xs(base, level='t').quantile(0.58)
hc = w.groupby('t').apply(lambda s: float((s <= z).mean()))
print(f"headcount on w, anchored at 58% in {base}:")
print(hc.round(3).to_string())

# The conventional measure, undeflated: nominal per-capita expenditure.
uga = ll.Country('Uganda')
xt = uga.food_expenditures().squeeze().groupby(['i', 't']).sum()
n = np.exp(uga.household_characteristics()['log HSize']).groupby(['i', 't']
pc = (xt / n.reindex(xt.index)).dropna()

med = pc.groupby('t').median()
print("median nominal per-capita food expenditure:")
print(med.round(0).to_string())
print(f"\nchange {waves[0]} -> {waves[1]}: "
      f"{100 * (med.loc[waves[1]] / med.loc[waves[0]] - 1):+.0f}%")
print(f"headcount on w, same interval: "
      f"{hc.loc[waves[0]]:.3f} -> {hc.loc[waves[1]]:.3f}")
```

One measure says welfare fell; the other says spending rose by a third. Reconciling them requires a deflator larger than the measured inflation over the period, which is to say: the conventional answer depends entirely on a number the survey does not contain.

5.8 A risk-sharing test

```
import statsmodels.api as sm

sh = uga.shocks()
print(sh.index.get_level_values('Shock').unique().tolist())

# One indicator per shock type, per household-wave.
S = (sh.assign(hit=1.0).hit
       .groupby(['i', 't', 'Shock']).max()
       .unstack('Shock').fillna(0.0))

wi = w.droplevel('m') if 'm' in (w.index.names or []) else w
df = pd.concat([wi.rename('w'), S], axis=1).dropna(subset=['w'])
df[S.columns] = df[S.columns].fillna(0.0)

# Keep the shocks that are common enough to estimate.
keep = [c for c in S.columns if df[c].mean() > 0.02]
print(f"{len(keep)} shock types with >2% incidence")

D = pd.get_dummies(df.index.get_level_values('t'), prefix='t',
                   drop_first=True, dtype=float)
D.index = df.index
X = pd.concat([df[keep], D], axis=1)
Z = pd.concat([df.w, X], axis=1)
Zd = Z - Z.groupby(level='i').transform('mean')      # household effects

res = sm.OLS(Zd.w, Zd.drop(columns='w')).fit(
    cov_type='cluster', cov_kwds={'groups': Zd.index.get_level_values('i')})
print(res.summary().tables[1])
```

Read the shock coefficients, not the time dummies. Under full risk sharing every one of them is zero. Where they are not, that household bore the shock itself.

And then read what is missing. A drought that struck the whole country in a given wave contributes nothing to these estimates: the time dummy took it. Whatever this regression says about insurance, it says it only about the shocks that hit some households and not others in the same period.

5.9 Three more places it breaks

- **Short panels.** $y_{it} = \rho y_{i,t-1} + \alpha_i + \epsilon_{it}$ cannot be estimated by within: demeaning correlates the lagged dependent variable with the demeaned error (Nickell bias, order $1/T$). At $T = 3$ the bias in $\hat{\rho}$ can exceed 30%, downward. Arellano–Bond instruments in differences, and is weak precisely when ρ is near one.

- **Measurement error.** Recall error grows with the window and the item count, and it is not classical. Differencing removes signal and keeps noise, so fixed effects can make things worse, not better.

- **Endogeneity.** Land, labour supply, migration, schooling, adoption — all chosen. Fixed effects handle time-invariant selection only. Panel data does not solve endogeneity; it solves one special case of it.

5.10 Where it breaks

It is worth ending on this note because the temptation, having spent four sessions learning to handle these data carefully, is to over-claim with them. The instinct runs the wrong way: careful handling of the design makes the descriptive estimates trustworthy, and does nothing whatever for the causal ones.

The honest position is that household surveys are the best instrument we have for saying who is poor, where, by how much, and how that has changed — and that these are important questions which most of the profession's methodology is not aimed at. A well-constructed poverty profile with defensible standard errors is a more useful contribution than a badly-identified regression, and it is also harder to do. Session 5's purpose is to leave you willing to do the first and reluctant to do the second.

5.11 The capstone

Pick a country — `lsms_library.countries()` lists them — and produce, in about ninety minutes:

1. one paragraph on the design: frame, stages, strata, weights, structure
2. a consumption aggregate, or a defence of using the one provided
3. one weighted, correctly-clustered estimate you are willing to defend
4. one plot

5. a notebook that runs from a clean kernel, top to bottom

Five minutes each to present. The audience's job is to ask what the design lets you claim.

5.12 What counts as reproducible

- runs top to bottom in a fresh kernel, in order
- no manual steps, no "then I edited the CSV"
- data pulled by code, not sitting in a folder
- versions recorded — `lsms_library.__version__`, `pandas.__version__`
- someone else's machine, not just yours

This is not bureaucracy. It is the only way you will be able to answer a referee two years from now.

5.13 Setting up your capstone

```
import lsms_library as ll

# Everything the library knows about
ll.countries()

# Which tables exist where?  'coverage' is a country-by-table matrix.
# refresh='coverage' reads the declarations directly, so it does not need a
# prebuilt snapshot on disk.
cov = ll.coverage(refresh='coverage')
cov.head(20)

# Pick one and look before you leap
c = ll.Country('Uganda')          # <- change me
print(c.waves)
print(c.data_scheme)
```

5.14 A reproducibility check

Put this at the top of the notebook you present.

```
import sys, platform
from importlib.metadata import version
import lsms_library, pandas, numpy, statsmodels

print(f"python        {sys.version.split()[0]}  ({platform.system()})")
print(f"lsms_library  {lsms_library.__version__}")
print(f"CFEDemands    {version('CFEDemands')}")
print(f"pandas        {pandas.__version__}")
print(f"numpy         {numpy.__version__}")
print(f"statsmodels   {statsmodels.__version__}")
```

If you have edited a country's configuration or scripts, the parquet cache will be stale, and there is no automatic staleness check:

```
# Force a rebuild for one country (slow; only when you have changed a confi
# !LSMS_NO_CACHE=1 python -c "import lsms_library as ll; ll.Country('Uganda
```

5.15 Exercises

The capstone is the exercise. Two suggestions if you finish early:

1. Take your estimate from (3) and compute it a second way — a different aggregate, a different scale, a different clustering level. Report the range. That range, not the standard error, is your real uncertainty.

2. Hand your notebook to the person next to you and have them run it from a clean kernel. Fix whatever breaks. That is the exercise.